Module 3:

Working with Design Patterns and Anti-patterns

Module 3: Working with Design Patterns and Anti-patterns

Module Focus

Learning Objectives:

- Understand how design patterns are applied in real software designs.
- Understand MVC as a design pattern for separating user-interface responsibilities.
- Apply general design principles to evaluate software structure.
- Analyze and critique code that exhibits poor design.
- Recognize anti-patterns and code smells before they make a system difficult to maintain.
- Use refactoring to improve internal code structure without changing external behavior.

- Key idea: Good design is not only about selecting a pattern; it is also about recognizing when a design has become unnecessarily coupled, complex, duplicated, or difficult to maintain.

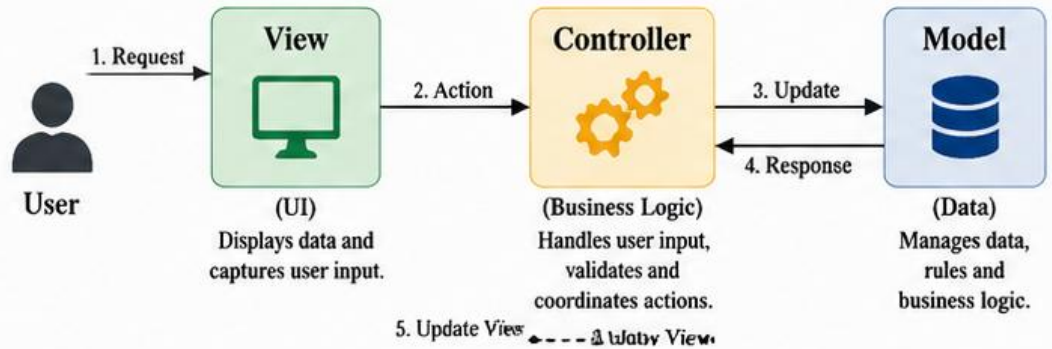
Module 3: Working with Design Patterns and Anti-patterns

Key idea: Good design is not only about selecting a pattern;
it is also about recognizing when a design has become
unnecessarily coupled, complex, duplicated, or difficult to maintain.

Module Focus

- Understand how design patterns are applied in real software designs.
- Understand MVC as a design pattern for separating user-interface responsibilities.
- Apply general design principles to evaluate software structure.
- Analyze and critique code that exhibits poor design.
- Recognize anti-patterns and code smells before they make a system difficult to maintain.
- Use refactoring to improve internal code structure without changing external behavior.

MVC Design Pattern



MVC separates concerns: *View* handles ***presentation***, *Controller* handles interaction, and ***Model*** **handles *data*** and logic.

General Design Principles

Single Responsibility Principle (SRP)
A class should have one reason to change.

Open/Closed Principle (OCP)
Open for extension, closed for modification.

Liskov Substitution Principle (LSP)
Subtypes must be substitutable for their base types.

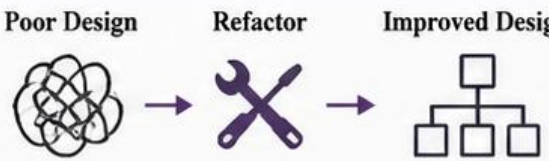
Interface Segregation Principle (ISP)
Clients should not be forced to depend on interfaces they don't use.

Dependency Inversion Principle (DIP)
Depend on abstractions, not on concretions.

Anti-patterns / Code Smells

- **Spaghetti Code:** Unstructured flow, hard to follow.
- **God Object:** A class that knows or does too much.
- **Duplicate Code:** Same code repeated in many places.
- **Long Method:** Method is too long, does too many things.
- **Tight Coupling:** Classes depend heavily on each other.

Refactoring

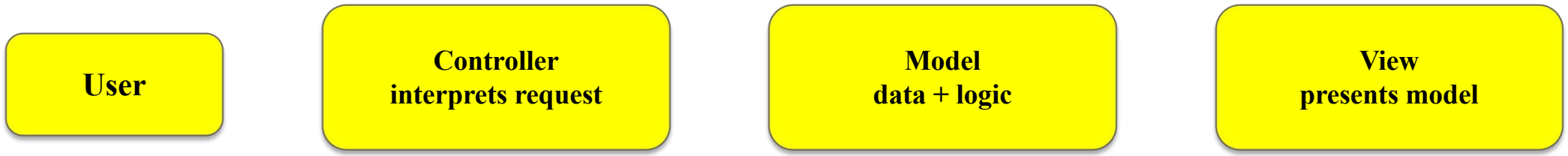


Improve *internal structure* without changing *external behavior*.

Good software design improves understandability, maintainability, testability, and scalability.
Always review, refactor, and evolve your design continuously.

MVC Design Pattern — Overview

- ✓ **The Model–View–Controller (MVC) pattern divides the responsibilities of a system that offers a user interface into three parts: Model, View, and Controller.**
- ✓ **Model:** Contains the underlying **data and logic** that users want to see and manipulate through an interface.
- ✓ **View:** Allows **users** to see all or part of the model.
- ✓ **Controller:** Interprets **user requests and changes** the model.
- ✓ **Main Goal:** Separate responsibilities so that user-interface presentation, request handling, and application data/logic are not unnecessarily mixed together.



Model–View–Controller (MVC) Design Pattern

What is MVC?

The Model–View–Controller (MVC) pattern divides the responsibilities of a system that offers a user interface into three parts: Model, View, and Controller.

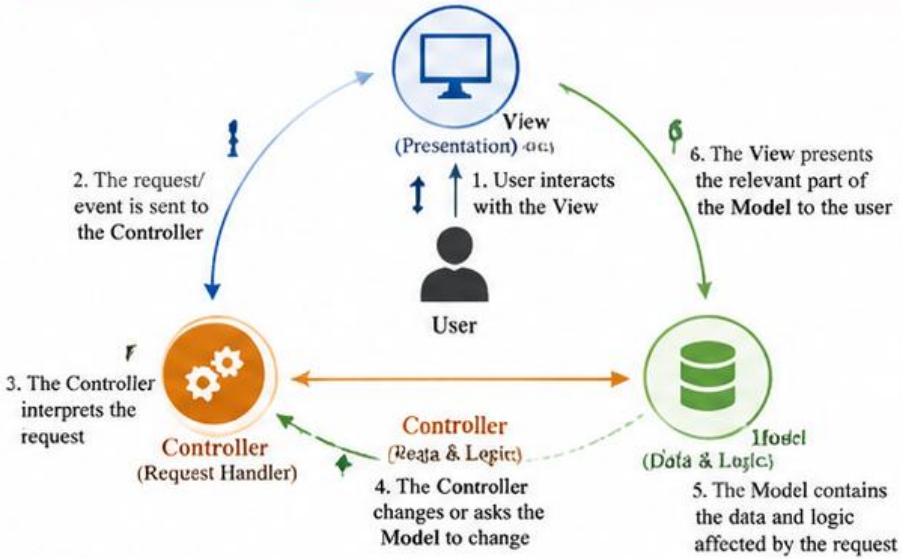
Model: Contains the underlying **data** and **logic** that users want to see and manipulate through an interface.

View: Allows **users** to see all or part of the model.

Controller: Interprets **user requests** and changes the model.

Main Goal:
Separate responsibilities so that user-interface presentation, request handling, and application data/logic are not unnecessarily mixed together.

MVC Interaction Flow



Result:
Each part has a clearer responsibility, making the system easier to **understand** and **modify**.

Scenario: A User Clicks "Add Student"

1. **View:** Displays the student form and collects the user's input.
2. **Controller:** Receives the ***Add Student*** request, validates/coordinates the request, and asks the Model to perform the operation.
3. **Model:** Represents **student data** and performs the required application/data operations.
4. **View:** Displays the updated student list or confirmation.

Separation of Responsibilities
- View should focus on presentation.
- Controller should coordinate requests.
- Model should contain the underlying data and logic.

PHP Laravel MVC vs Java MVC (Comparison)

Aspect	PHP Laravel MVC	Java MVC (e.g., Spring MVC)
Framework Type	Full-stack MVC framework	MVC implemented in enterprise frameworks
Routing / Request Handling	Handled by Laravel **Router** (routes/web.php, **routes/api.php**)	Handled by **DispatcherServlet** and @RequestMapping
Controller	PHP classes (extend Controller), methods return View/Response	Java classes with @Controller or @RestController
View	Blade templates (server-side rendering)	JSP, Thymeleaf, or other view technologies
Model	Eloquent ORM (ActiveRecord pattern)	**JPA/Hibernate (Entity pattern)**
Data Binding	Automatic via Request validation and Eloquent	Automatic via **Model** attributes and data binding
Configuration	Convention over Configuration	More configuration (XML/Java Config/Annotations)
Example Flow	Route → Controller → Model (Eloquent) → View (Blade)	Request → Controller → Model (Service/Repository) → View
Use Cases	Rapid web development, small to medium applications	Large-scale, enterprise, highly modular applications

Why MVC?

- Separation of concerns (clearly defined roles)
- Easier maintenance and modifications
- Better testability (each part can be tested independently)
- Supports team collaboration (frontend, backend, database teams can work separately)

MVC — Request and Response Flow

Typical interaction flow:

1. User interacts with the View.
2. The request/event is sent to the Controller.
3. The Controller interprets the request.
4. The Controller changes or asks the Model to change.
5. The Model contains the data and logic affected by the request.
6. The View presents the relevant part of the Model to the user.

Result: Each part has a clearer responsibility, making the system easier to understand and modify.



MVC — Simple Example: Student Management System

- **Scenario:** A user clicks “Add Student”.
- **View:** Displays the student form and collects the user's input.
- **Controller:** Receives the Add Student request, validates/coordinates the request, and asks the Model to perform the operation.
- **Model:** Represents student data and performs the required application/data operations.
- **View:** Displays the updated student list or confirmation.
- **Separation of responsibilities:**
  - View should focus on presentation.
  - Controller should coordinate requests.
  - Model should contain the underlying data and logic.

General Design Principles — Why They Matter

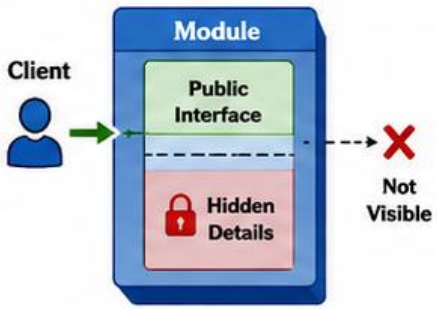
- Design principles provide guidance for creating reusable, flexible, understandable, and maintainable software.
- **Information Hiding:** Modules should expose the minimum information needed to use them correctly and hide the rest.
- **Separation of Concerns:** Organize software so different concerns are separated into different sections and addressed effectively.
- **Dependency Inversion:** High-level modules should depend on high-level generalizations rather than low-level implementation details.
- **Open/Closed Principle:** Classes should be open for extension but closed to change.
- **Interface Segregation:** A class should not be forced to depend on methods it does not use.
- **Coupling Objects Principle:** Classes should achieve code reuse through aggregation rather than inheritance.

General Design Principles — Why They Matter

Design principles provide guidance for creating reusable, flexible, understandable, and maintainable software.

1 Information Hiding

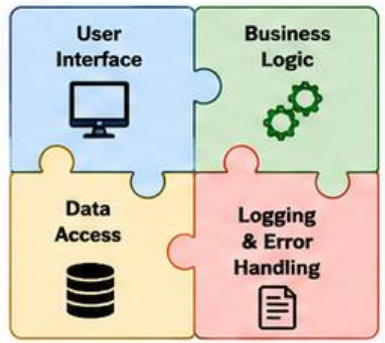
Modules should expose the minimum information needed to use them correctly and hide the rest.



Expose what is necessary, hide what is not.

2 Separation of Concerns

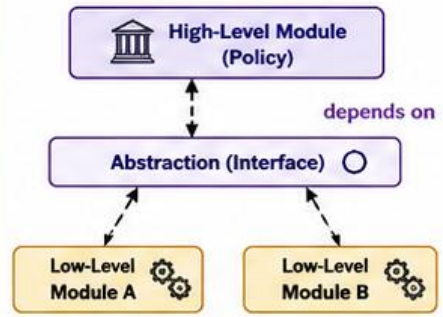
Organize software so different concerns are separated into different sections and addressed effectively.



Each concern changes for a different reason.

3 Dependency Inversion

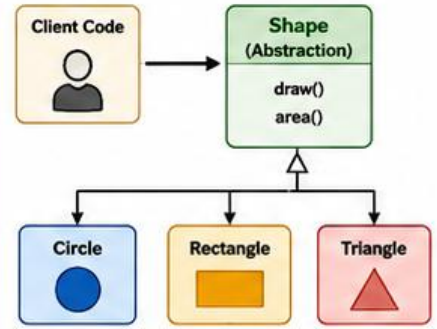
High-level modules should depend on high-level generalizations rather than low-level implementation details.



Depend on abstractions, not on concretions.

4 Open/Closed Principle

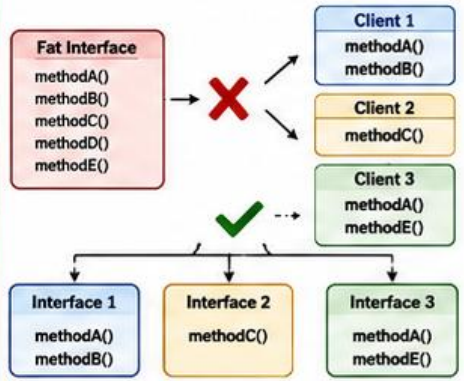
Classes should be open for extension but closed to change.



Extend behavior via new classes, not by modifying existing code.

5 Interface Segregation

A class should not be forced to depend on methods it does not use.



Many small interfaces are better than one large interface.

General Design Principles — DRY and Least Knowledge

6 Don't Repeat Yourself (DRY)

- ✓ Write programs that perform the same tasks with less repeated code.
- ✓ Reuse shared blocks of code so systems are easier to maintain.

Without DRY (Duplicated Code)

```
void totalOrder1() {
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}
void totalOrder2() {
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}
```

With DRY (Reusable Code)

```
int calculateTotal(){
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}

void totalOrder1(){
  int sum = calculateTotal();
}

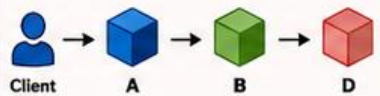
void totalOrder2(){
  int sum = calculateTotal();
}
```

One change in one place — everywhere it is used.

7 Principle of Least Knowledge / Law of Demeter

- ✓ Classes should know about and interact with as few other classes as possible.
- ✓ Avoid unnecessary navigation through chains of dependent objects.

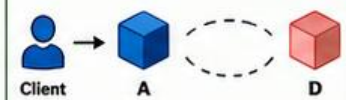
Violation (Too Much Knowledge)




```
client.getA().getB().getC().getD().doWork();
```

Tight coupling, fragile code

Good (Least Knowledge)




```
client.getA().doWork();
```

Loose coupling, easier to maintain

Talk only to your immediate friends.

Why these principles matter:

- Less duplication means fewer places to update.
- Less knowledge reduces coupling.
- Simpler dependencies make testing and refactoring easier.
- Results in software that is more maintainable, flexible, and scalable.



Good design is not about adding more code. It is about organizing code smartly. | Better Design ⇒ Better Quality ⇒ Better Software

General Design Principles — DRY and Least Knowledge

✓ Don't Repeat Yourself (DRY)

- Write programs that perform the same tasks with less repeated code.
- Reuse shared blocks of code so systems are easier to maintain.

✓ Principle of Least Knowledge / Law of Demeter

- Classes should know about and interact with as few other classes as possible.
- Avoid unnecessary navigation through chains of dependent objects.
- **Why these principles matter:**
  - Less duplication means fewer places to update.
  - Less knowledge reduces coupling.
  - Simpler dependencies make testing and refactoring easier.

How to Analyze and Critique a Design — Checklist

Step 1: Identify Responsibilities

Ask: What does each class actually do? Does it have one clear purpose?

Step 2: Identify Dependencies

Ask: Which classes know about each other? Are there unnecessary dependencies or cycles?

Step 3: Check Coupling and Cohesion

Ask: Are responsibilities tightly connected inside a class, and are unrelated responsibilities separated?

Step 4: **Look for Code Smells:** Check for duplication, long methods, large classes, message chains, primitive obsession, switch statements, and inappropriate intimacy.

Step 5: **Check Principles:** Review information hiding, separation of concerns, DRY, least knowledge, dependency inversion, open/closed, and interface segregation.

Step 6: **Refactor:** Improve the internal structure while preserving external behavior.

Analyzing and Critiquing Code — What Is Bad Design?

When reviewing code, ask:

1. Does one class have too many unrelated responsibilities?

2. Are responsibilities placed in the wrong class?

3. Are classes tightly coupled through unnecessary dependencies?

4. Is the same logic duplicated in multiple locations?

5. Are long methods, long parameter lists, or long call chains making behavior difficult to understand?

6. Are primitive values being used where meaningful abstractions/classes should exist?

7. Are switch statements scattered throughout the system?

8. Is the code generalized for requirements that do not yet exist?

9. Does inheritance force subclasses to receive behavior they do not need?

Analyzing and Critiquing Code — What Is Bad Design?

When reviewing code, ask these important questions:

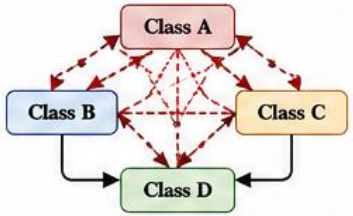
1 Does one class have too many unrelated responsibilities?



2 Are responsibilities placed in the wrong class?

Wrong Class ✗ → Right Class ✓

3 Are classes tightly coupled through unnecessary dependencies?



4 Is the same logic duplicated in multiple locations?



5 Are long methods, long parameter lists, or long call chains making behavior difficult to understand?

```
method(a, b, c, d, e,
       f, g, h, i, j, k, l,
       m, n, o, p, q, r)
{
  ... many lines of code ...
  methodA().methodB()
           .methodC()
           .methodD();
}
```

6 Are primitive values being used where meaningful abstractions/classes should exist?

Primitive Obsession

```
String name
int age
int type
double amount
```

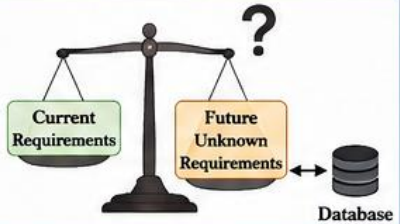
Better: Use Meaningful Classes

```
Person
Age
OrderType
Money
```

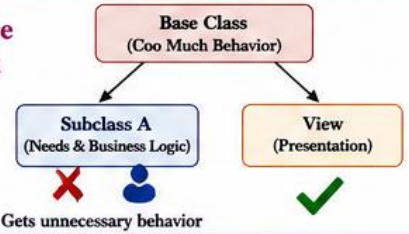
7 Are switch statements scattered throughout the system?

```
switch(type) {
  case A: ...
  case B: ...
  case C: ...
  default: ...
}
```

8 Is the code generalized for requirements that do not yet exist?



9 Does inheritance force subclasses to receive behavior they do not need?

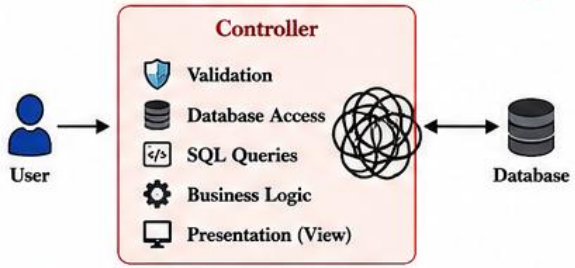


Code Critique Example — Multiple Responsibilities

Critique:

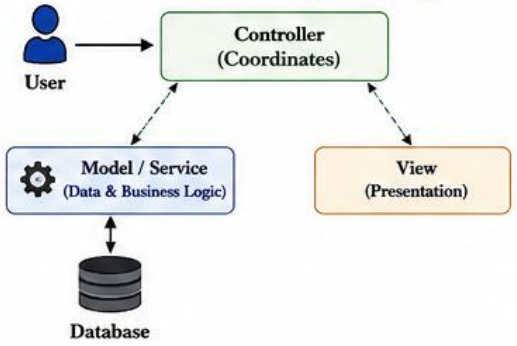
- ✗ Controller handles validation, database access, SQL, and presentation.
- ✗ Responsibilities are mixed together.
- ✗ Changing the database can affect the controller.
- ✗ Changing the presentation can affect the controller.
- ✗ Difficult to test each responsibility independently.

Problem: Everything in One Class (Controller)



Hard to maintain, hard to change, hard to test!

Better Direction: Separate Responsibilities



- ✓ Controller coordinates.
- ✓ Model/service handles data and application logic.
- ✓ View handles presentation.
- ✓ Each part is easier to understand, change, and test.

★ Good design leads to clean, maintainable, flexible, and testable software. → Bad design leads to complexity, bugs, and cost. ★

Code Analysis/ Code Critique Example — Multiple Responsibilities

```
class StudentController {

void addStudent(String name, String email) {
                // validate input
   if (name == null || name.isEmpty()) return;

          // create database connection
Connection db = DriverManager.getConnection(...);

                // SQL operation
                  String sql =
   "INSERT INTO students(name,email) VALUES(?,?)";

                // execute SQL
                    // ...

             // format HTML response
               System.out.println(
    "<html><body>Student Added</body></html>");
                    }
                  }
```

✓**Critique:**

- ✓Controller handles validation, database access, SQL, and presentation.
- ✓Responsibilities are mixed together.
- ✓Changing the database can affect the controller.
- ✓Changing the presentation can affect the controller.
- ✓Difficult to test each responsibility independently.

✓**Better direction:**

- ✓Controller coordinates.
- ✓Model/service handles data and application logic.
- ✓View handles presentation.

Code Analysis/ Code Critique Example — Scattered Switch Statements

```
double calculatePrice(String type) {
            switch(type) {
        case "REGULAR": return 100;
        case "PREMIUM": return 150;
            default: return 0;
                }
            }

String getDescription(String type) {
            switch(type) {
     case "REGULAR": return "Regular";
     case "PREMIUM": return "Premium";
         default: return "Unknown";
                }
            }

 double getDiscount(String type) {
            switch(type) {
       case "REGULAR": return 0.05;
       case "PREMIUM": return 0.15;
            default: return 0;
                }
            }
```

✓Critique:

- ✓The same type information is checked repeatedly.
- ✓Adding a new type requires changing several methods.
- ✓The design can become difficult to maintain.
- ✓This is the kind of situation where polymorphism may reduce scattered conditionals.

✓Possible refactoring:

- ✓Create a common Product/Customer/Plan abstraction.
- ✓Move price, description, and discount behavior into the appropriate concrete classes.

Code Analysis/ Code Critique Example — Primitive Obsession

```
void createOrder(
 String customerName,
String customerEmail,
   String country,
  String currency,
   String street,
    String city,
String postalCode) {

// many operations using raw Strings
}
```

- ✓ **Potential smell:**
  - ✓ Many Strings represent different concepts.
  - ✓ The code does not clearly express the domain abstractions.
  - ✓ Validation and behavior may be duplicated across the system.
- ✓ **Possible improvement:**
  - ✓ Create meaningful types such as Customer, EmailAddress, Address, PostalCode, and Money where appropriate.
  - ✓ Encapsulate validation and related behavior in those abstractions.
- ✓ Result: The design communicates the domain more clearly and reduces primitive obsession.

Anti-patterns and Code Smells

Anti-patterns: Patterns of bad code that commonly emerge in software design; they are also described as code smells.

Code smells are warning signs rather than automatic proof that code is wrong.



Why Identify Them?

- They help identify bad code and bad design early.
- They indicate places where refactoring may improve reuse, flexibility, and maintainability.
- Regular code review can prevent small problems from becoming major design problems.

Common Code Smells (Examples)

Long Method	Method is too long and does too many things.
Large Class	Class has too many responsibilities and too many fields.
Duplicate Code	Same code appears in multiple places.
`f( a, b, c, d, e, f, g )` Long Parameter List	Method has too many parameters, hard to understand and use.
Feature Envy	Method uses more data from other classes than its own.
Data Clumps	Groups of data travel together but have no structure.

Important

A smell should be investigated in context. Some structures may be appropriate when there is a valid design reason.



Understand the reason.
Make the right decision.

Anti-patterns and Code Smells

- ✓ **Anti-patterns: Patterns of bad code that commonly emerge in software design; they are also described as code smells.**
- ✓ Code smells are warning signs rather than automatic proof that code is wrong.
- ✓ **Why Identify Them?**
  - ✓ They help identify bad code and bad design early.
  - ✓ They indicate places where refactoring may improve reuse, flexibility, and maintainability.
  - ✓ Regular code review can prevent small problems from becoming major design problems.
- ✓ **Important:** A smell should be investigated in context. Some structures may be appropriate when there is a valid design reason.

Common Anti-patterns / Code Smells in This Module

Large Class — a class that is too long; also called a God, Blob, or Black Hole class.

Long Method — a method containing excessive logic.

Duplicated Code — similar code blocks appear in multiple places with slight differences.

Data Class — a class containing mostly data and little real functionality.

Data Clumps — groups of data repeatedly appear together as fields or method parameters.

Divergent Change — one class must be changed in many different ways for many different reasons.

Shotgun Surgery — one requirement change requires changes across many classes.

Feature Envy — a method is more interested in another class's details than its own class.

Long Parameter Lists — methods require too many parameters.

Comments as a smell — excessive comments may indicate that comments are being used to explain or cover a difficult design.

Code Smells — Inappropriate Intimacy and Message Chains

- ✓ **Inappropriate Intimacy:** Occurs when two classes depend too much on each other through two-way communication.
  - ✓ If methods in one class call the other and vice versa, consider removing the cycle.
  - ✓ Factor methods so both classes can use another class, or at least make communication one-way.
- ✓ **Message Chains:** Long chains of calls can create rigidity and complexity and make code harder to test independently.
  - ✓ They can violate the Law of Demeter when objects know too much about other objects' dependencies.
  - ✓ Navigation through dependencies should not be unnecessarily baked into objects.
- ✓ **Important qualification:** A message chain is not automatically bad if it returns a limited set of objects and follows the Law of Demeter.

Code Smells — Primitive Obsession and Switch Statements

- ✓ **Primitive Obsession:**
  - ✓ Occurs when the design relies too much on built-in types such as int, long, float, and String.
  - ✓ Overuse can indicate that meaningful abstractions/classes have not been identified.
  - ✓ High-level use of primitives may hide important domain concepts.
- ✓ **Switch Statements:**
  - ✓ Scattered switch statements become a problem because changing one case may require finding and updating many other switches.
  - ✓ When conditionals repeatedly check type codes, consider whether polymorphism could express the behavior more clearly.
- ✓ **Better direction:** Identify missing abstractions and move behavior into appropriate classes rather than encoding everything as primitive values or repeated conditionals.

Code Smells — Speculative Generality and Refused Request

Speculative Generality

- Creating a superclass, interface, or code that is not currently needed but may be useful someday.
- It introduces generality that may not help and can over-engineer the system.
- Just-in-Time Design suggests designing only what is needed for the current requirements/iteration.
- If generalization becomes necessary later, refactor when the requirement is real.

Refused Request

- A subclass inherits behavior that it does not need or use.
- This may indicate that the subclass is not appropriate for the superclass.
- Consider a standalone class or move behavior to the subclasses that actually need it.

Example-1: Code Smells — Problem

Problem: A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach

Solution: Likely Code Smell: Long Method

The method contains **100 lines of code** and performs several different responsibilities:

- Input validation
- Database access
- Calculations
- Report generation

This indicates that the method has **too many responsibilities**, making it difficult to understand, test, maintain, and modify.

Appropriate Refactoring Approach: Extract Method

The large method should be divided into several smaller, focused methods, where each method performs one specific task.

The likely code smell is **Long Method** because the method contains about 100 lines and performs several different responsibilities such as validation, database access, calculation, and report generation. The appropriate refactoring technique is **Extract Method**. The large method should be divided into smaller methods such as validateStudent(), getStudentMarks(), calculateAverage(), and generateReport(). This improves **readability, maintainability, reusability, and testability**.

Code Smells — Problem

Before Refactoring Long Method:

```
public void processStudent() {

   // Validation
   if (studentName == null || studentName.isEmpty()) {
      System.out.println("Invalid student name");
      return;
   }

   if (studentId <= 0) {
      System.out.println("Invalid student ID");
      return;
   }

   // Database access
   Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/university",
      "root",
      "password"
   );

   PreparedStatement ps = con.prepareStatement(
      "SELECT marks FROM students WHERE id = ?"
   );
   ps.setInt(1, studentId);

   ResultSet rs = ps.executeQuery();

   // Calculation
   double total = 0;
   int count = 0;

   while (rs.next()) {
      total += rs.getDouble("marks");
      count++;
   }

   double average = total / count;

   // Report generation
   System.out.println("Student: " + studentName);
   System.out.println("Student ID: " + studentId);
   System.out.println("Average Marks: " + average);

   // Many more lines...
}
```

After Refactoring:

```
public void processStudent() {

   if (!validateStudent()) {
      return;
   }

   List<Double> marks = getStudentMarks();

   double average = calculateAverage(marks);

   generateReport(average);
}
```

The responsibilities are now separated:

```
private boolean validateStudent() {
   if (studentName == null ||
studentName.isEmpty()) {
      System.out.println("Invalid student name");
      return false;
   }

   if (studentId <= 0) {
      System.out.println("Invalid student ID");
      return false;
   }

   return true;
}

private List<Double> getStudentMarks() {
   List<Double> marks = new ArrayList<>();

   // Database access code
   // Execute SQL query
   // Retrieve marks

   return marks;
}
```

```
private double calculateAverage(List<Double> marks) {
   if (marks.isEmpty()) {
      return 0;
   }
   double total = 0;

   for (double mark : marks) {
      total += mark;
   }

   return total / marks.size();
}
```

```
private void generateReport(double average) {
   System.out.println("Student: " + studentName);
   System.out.println("Student ID: " + studentId);
   System.out.println("Average Marks: " + average);
  }
```

Item	Answer
Code Smell	**Long Method**
Problem	One method performs too many tasks
Refactoring Technique	**Extract Method**
Main benefit	Improves readability, maintainability, and testability
Extracted methods	validateStudent(), getStudentMarks(), calculateAverage(), generateReport()

Code Smells — Speculative Generality and Refused Request

Speculative Generality

- Creating a superclass, interface, or code that is not currently needed but may be useful someday.
- It introduces generality that may not help and can over-engineer the system.
- Just-in-Time Design suggests designing only what is needed for the current requirements/iteration.
- If generalization becomes necessary later, refactor when the requirement is real.

Refused Request

- A subclass inherits behavior that it does not need or use.
- This may indicate that the subclass is not appropriate for the superclass.
- Consider a standalone class or move behavior to the subclasses that actually need it.

MVC Anti-pattern — When Responsibilities Are Mixed

A common poor MVC design occurs when one class attempts to perform the responsibilities of Model, View, and Controller simultaneously.

Symptoms:

- Database queries inside UI code.
- HTML/UI formatting inside data/model classes.
- Business rules directly inside event-handling code.
- Controllers that become extremely large and perform every operation themselves.

Problems:

- High coupling.
- Difficult testing.
- Difficult maintenance.
- Changes in one concern affect unrelated concerns.

Better approach: Preserve separation of concerns and assign each responsibility to the appropriate MVC component.

Example-2: If a System Suffering from a Large Class and Duplicated Code

1. Identify the Code Smells

There are two major code smells:

① **Large Class** – One class contains too many responsibilities.

② **Duplicated Code** – The same or very similar code is repeated in multiple methods.

Example Scenario

Suppose we have a StudentManager class that handles:

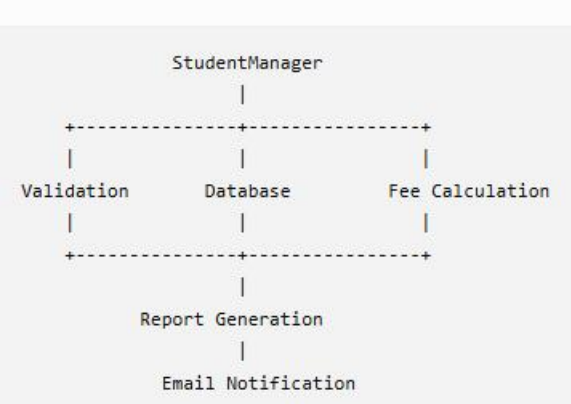
① Student information
② Student validation
③ Database operations
④ Fee calculation
⑤ Report generation
⑥ Email notification

This violates the **Single Responsibility Principle (SRP)**.

2. Before Refactoring

① A poorly designed class might very long code

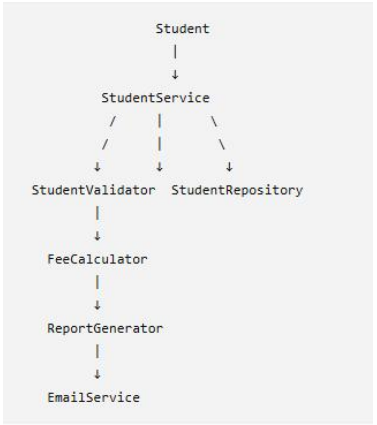
② For Example: The StudentManager class is doing **too many jobs**:



It also contains duplicated validation and fee-calculation code.

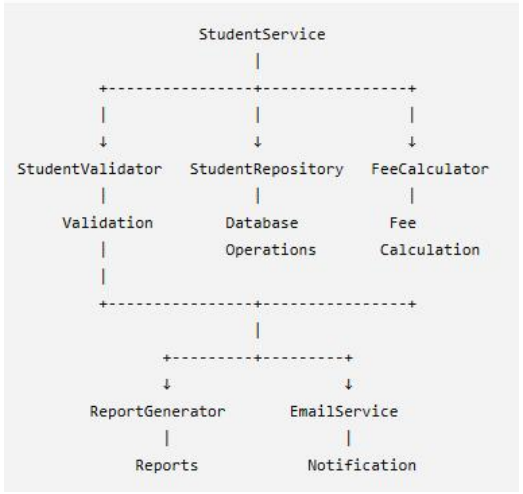
3. Refactored Design

① We can redistribute the responsibilities into several classes:



Class	Responsibility
Student	Store student information
StudentValidator	Validate student information
StudentRepository	Database operations
FeeCalculator	Calculate tuition fees
ReportGenerator	Generate student reports
EmailService	Send email notifications
StudentService	Coordinate the overall student operation

6. Final Design: The final structure can be represented as



If a System Suffering from a Large Class and Duplicated Code: Java Example

Before Refactoring Long Method:

```
public class StudentManager {

    private String name;
    private int studentId;
    private double tuitionFee;

    public void registerStudent() {
        // Student validation
        if (name == null || name.isEmpty()) {
            System.out.println("Invalid student name");
            return;
        }
        if (studentId <= 0) {
            System.out.println("Invalid student ID");
            return;
        }
        // Database operation
        System.out.println("Saving student to database...");
        // Fee calculation
        double fee = tuitionFee;
        if (fee > 50000) {
            fee = fee * 0.90;   // 10% discount
        }
        System.out.println("Final fee: " + fee);
        // Report generation
        System.out.println("Student Report");
        System.out.println("Name: " + name);
        System.out.println("ID: " + studentId);
        // Email notification
        System.out.println("Sending email to student...");
    }

    public void updateStudent() {
        // Same validation code - DUPLICATED
        if (name == null || name.isEmpty()) {
            System.out.println("Invalid student name");
            return;
        }
        if (studentId <= 0) {
            System.out.println("Invalid student ID");
            return;
        }
        // Database operation
        System.out.println("Updating student in database...");
        // Same fee calculation - DUPLICATED
        double fee = tuitionFee;

        if (fee > 50000) {
            fee = fee * 0.90;
        }
        System.out.println("Updated fee: " + fee);
        // Email notification
        System.out.println("Sending email to student...");
    }
}
```

If a System Suffering from a Large Class and Duplicated Code: Java Example

After Refactoring Long Method:

```
public class Student {

   private int studentId;
   private String name;
   private double tuitionFee;

   public Student(int studentId, String name, double tuitionFee){
      this.studentId = studentId;
      this.name = name;
      this.tuitionFee = tuitionFee;
   }

   public int getStudentId() {
      return studentId;
   }

   public String getName() {
      return name;
   }

   public double getTuitionFee() {
      return tuitionFee;
   }
}
```

```
public class StudentRepository {
   public void save(Student student) {
      System.out.println("Saving student to database...");
   }
   public void update(Student student) {
      System.out.println("Updating student in database...");
   }
}
```

```
public class StudentValidator {
   public boolean isValid(Student student) {
      if (student.getName() == null ||
         student.getName().isEmpty()) {
         return false;
      }
      if (student.getStudentId() <= 0) {
         return false;
      }
      return true;
   }
}
```

```
public class ReportGenerator {
   public void generate(Student student, double fee) {
      System.out.println("Student Report");
      System.out.println("Name: " + student.getName());
      System.out.println("ID: " + student.getStudentId());
      System.out.println("Final Fee: " + fee);
   }
}
```

```
public class FeeCalculator {
   public double calculateFee(double tuitionFee) {
      if (tuitionFee > 50000) {
         return tuitionFee * 0.90;
      }return tuitionFee;
   }
}
```

```
public class EmailService {
   public void sendEmail(Student student) {
      System.out.println(
         "Sending email to " + student.getName()
      );
   }
}
```

```
public class StudentService {
   private StudentValidator validator;
   private StudentRepository repository;
   private FeeCalculator calculator;
   private ReportGenerator reportGenerator;
   private EmailService emailService;

   public StudentService() {
      validator = new StudentValidator();
      repository = new StudentRepository();
      calculator = new FeeCalculator();
      reportGenerator = new ReportGenerator();
      emailService = new EmailService();
   }

   public void registerStudent(Student student) {
      if (!validator.isValid(student)) {
         System.out.println("Invalid student information.");
         return;
      }
      repository.save(student);
      double fee =
         calculator.calculateFee(student.getTuitionFee());
      reportGenerator.generate(student, fee);
      emailService.sendEmail(student);
   }

   public void updateStudent(Student student) {
      if (!validator.isValid(student)) {
         System.out.println("Invalid student information.");
         return;
      }

      repository.update(student);
      double fee =
         calculator.calculateFee(student.getTuitionFee());
      reportGenerator.generate(student, fee);
      emailService.sendEmail(student);
   }
}
```

If a System Suffering from a Large Class and Duplicated Code: Java Example

After Refactoring Long Method:

```
public class StudentService {
    private StudentValidator validator;
    private StudentRepository repository;
    private FeeCalculator calculator;
    private ReportGenerator reportGenerator;
    private EmailService emailService;

    public StudentService() {

        validator = new StudentValidator();
        repository = new StudentRepository();
        calculator = new FeeCalculator();
        reportGenerator = new ReportGenerator();
        emailService = new EmailService();
    }
    public void registerStudent(Student student) {

        if (!validator.isValid(student)) {
            System.out.println("Invalid student information.");
            return;
        }
        repository.save(student);
        double fee =
            calculator.calculateFee(student.getTuitionFee());

        reportGenerator.generate(student, fee);
        emailService.sendEmail(student);
    }
    public void updateStudent(Student student) {
        if (!validator.isValid(student)) {
            System.out.println("Invalid student information.");
            return;
        }
        repository.update(student);
        double fee =
            calculator.calculateFee(student.getTuitionFee());
        reportGenerator.generate(student, fee);
        emailService.sendEmail(student);
    }
}
```

Problem	Refactoring Solution
Large Class	Extract Class / Responsibility redistribution
Duplicated Code	Extract Method / Move common logic to a single class
Too many responsibilities	Apply **Single Responsibility Principle (SRP)**
Repeated validation	Centralize in StudentValidator
Repeated fee calculation	Centralize in FeeCalculator
Database code mixed with business logic	Move to StudentRepository

Design Patterns vs. Anti-patterns (Code Smells)

Key Lesson: Do not apply patterns simply because they exist.
Choose a pattern because it solves a real design problem.
Likewise, do not automatically remove every code smell; **understand the context and trade-offs first.**

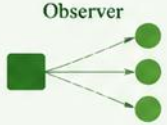
DESIGN PATTERN

Example Patterns

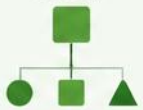
Singleton



Observer



Factory



Reusable solution
Provides a general, reusable solution to a recurring software design problem.

Improves communication
Offers a common vocabulary that helps developers communicate design ideas clearly.

Proven approach
Based on experience and proven in many successful projects.

Used intentionally
Applied when the context is appropriate and the problem fits the pattern.

ANTI-PATTERN / CODE SMELL

Recurring poor design
Represents a tendency toward poor design or implementation.

Warning sign
Signals that the code may be difficult to reuse, modify, test, or maintain.

Investigate the cause
Indicates deeper problems that should be understood and addressed.

Refactor when appropriate
Should be refactored when it hurts maintainability or increases complexity.

Common Code Smells

Long Method



Duplicate Code



Large Class



Large Coupling



Tight Coupling

When to Use a Design Pattern?

- When you recognize a recurring design problem.
- When it improves flexibility, maintainability, or extensibility.
- When the added complexity of the pattern is justified.

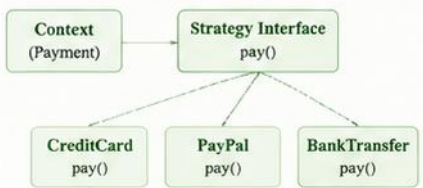


Good design is about making the right choice.

When to Investigate a Code Smell?

- When it appears frequently in the codebase.
- When it makes changes risky or hard to test.
- When it violates good design principles.
- When it affects readability or performance.

Example: Good Design Using a Pattern (Strategy Pattern)



- Open for extension (add new payment easily)
- Closed for modification (existing code unchanged)
- Easy to test and maintain
- Low coupling, high flexibility

Refactor to Improve

Example: Bad Design (Code Smell – Long Method & Tight Coupling)

```
public void proccesPayment(String type) {
   if (type.equals("card")) {
      // do card payment
   } else if (type.equals("paypal")) {
      // do paypal payment
   } else if (type.equals("bank")) {
      // do bank transfer
   }
   // Many more conditions...
}
```

- Hard to extend (changes require modification)
- Violates Single Responsibility
- Difficult to test
- High coupling and complexity
- Prone to bugs

Takeaway: Good software design comes from recognizing **the right pattern for the right problem** and addressing the real issues behind code smells—not by following rules blindly.

Design Patterns vs. Anti-patterns

Design Pattern

Reusable solution to a recurring software design problem.

Improves communication and provides a proven design approach.

Used intentionally when the context is appropriate.

Anti-pattern / Code Smell

Recurring poor design or implementation tendency.

Warns that code may be difficult to reuse, modify, test, or maintain.

Should be investigated and potentially refactored.

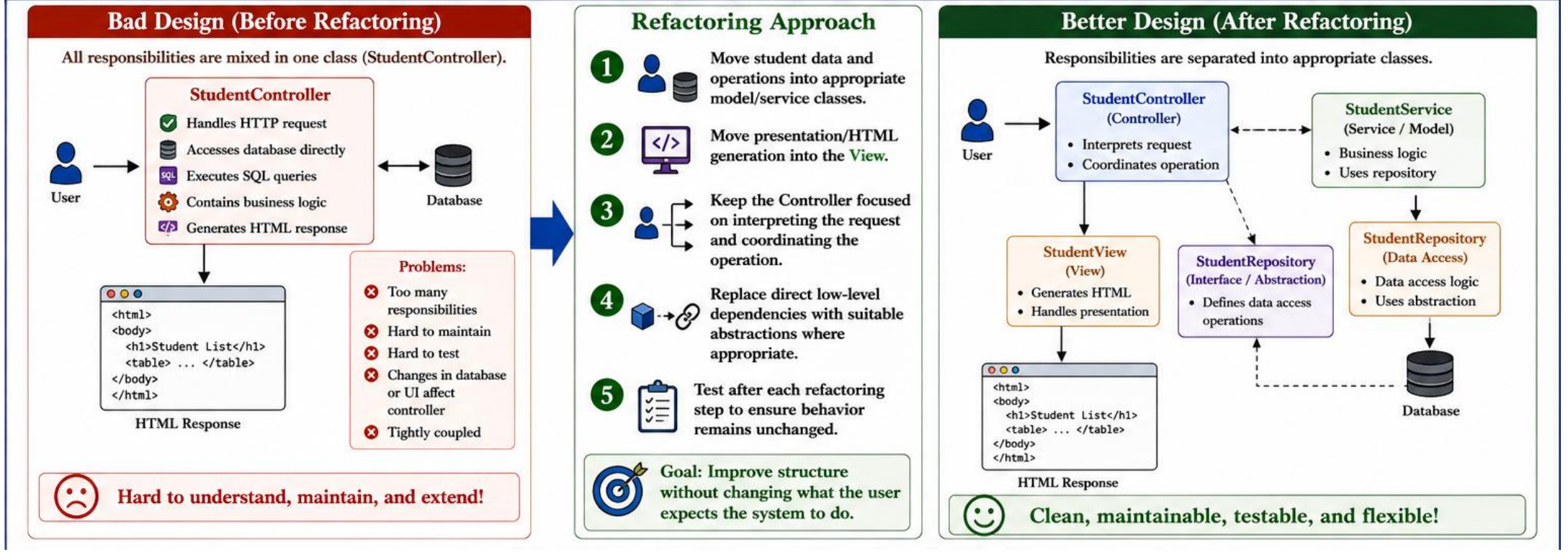
Key lesson:

Do not apply patterns simply because they exist.

Choose a pattern because it solves a real design problem.

Likewise, do not automatically remove every code smell; understand the context and trade-offs first.

From Bad Design to Better Design — Refactoring



From Bad Design to Better Design — Refactoring

Refactoring

Refactoring is the process of changing code so that its external behavior remains the same while its internal structure is improved.

Refactoring approach for the previous example:

1. Move student data and operations into appropriate model/service classes.
2. Move presentation/HTML generation into the View.
3. Keep the Controller focused on interpreting the request and coordinating the operation.
4. Replace direct low-level dependencies with suitable abstractions where appropriate.
5. Test after each refactoring step to ensure behavior remains unchanged.

Goal: Improve structure without changing what the user expects the system to do.

Goal

- Create software that remains reusable, flexible, understandable, and maintainable as requirements change.
- Key idea: Good design is not only about selecting a pattern; it is also about recognizing when a design has become unnecessarily coupled, complex, duplicated, or difficult to maintain.

Module 3 — Summary

- **MVC:** Separates a user-interface system into **Model, View, and Controller** responsibilities.
- **General Design Principles: Information hiding, separation of concerns, dependency inversion, open/closed, interface segregation, coupling objects, DRY, and least knowledge.**
- **Code Analysis:** Look for excessive responsibilities, **coupling, duplication, long methods, complex** dependencies, and inappropriate abstractions.
- **Anti-patterns / Code Smells**
  - Inappropriate intimacy, message chains, primitive obsession, switch statements, speculative generality, refused request, and many other smells.
- **Refactoring:** Improve internal structure while preserving external behavior.
- **Final Goal:** Create software that remains reusable, flexible, understandable, and maintainable as requirements change.

Software Design Concepts Summary with Dhaka Urban Management System (DUMS) Examples

Topic	Short Explanation	Application in DUMS (Dhaka Urban Management System)	Example in DUMS
MVC (Model–View–Controller)	Separates a user-interface system into Model, View, and Controller responsibilities.	Divides DUMS into Model (urban data & logic), View (citizen/ddmin interfaces), Controller (request processing).	Citizen submits a waste complaint → View: Complaint Form → Controller: Processes request → Model: Stores complaint in database → View shows status update.
Information Hiding	Hides internal implementation details and exposes only necessary interfaces.	Protects internal operations of urban service modules.	Traffic module provides *getTrafficStatus()* API without exposing how traffic prediction is calculated.
Separation of Concerns (SoC)	Divides a system into independent parts, each handling a specific responsibility.	Separates traffic, waste, emergency, payment, and citizen service modules.	Waste collection scheduling is handled by Waste Management Module, not mixed with traffic operations.
Dependency Inversion Principle (DIP)	High-level modules should depend on abstractions, not specific implementations.	Urban services communicate through common interfaces.	EmergencyService depends on NotificationService interface rather than directly depending on SMS or Email systems.
Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	New urban services can be added without changing existing modules.	Adding a Smart Parking Module without modifying Traffic Management code.
Interface Segregation Principle (ISP)	Clients should not depend on unnecessary methods.	Provides specific interfaces for different users and services.	Traffic Officer uses *TrafficControlInterface*, while Waste Manager uses *WasteCollectionInterface*.
Coupling Management	Reduces dependency between modules to improve flexibility.	DUMS modules communicate through APIs/services instead of direct dependency.	Traffic Management receives road information from a shared City Data Service instead of directly depending on Waste Management.
DRY (Don't Repeat Yourself)	Avoids duplicate code and logic.	Common functions are reused across modules.	A single **NotificationService** is used for complaints, emergency alerts, and service updates.
Least Knowledge Principle	A module should know only necessary information about other modules.	Limits unnecessary communication between urban modules.	Complaint Module only requests service status; it does not access internal Waste Management operations.

Code Analysis: What to Look For

- Excessive responsibilities
- High coupling between modules
- Code duplication
- Long methods and large classes
- Complex dependencies
- Inappropriate abstractions

Anti-patterns / Code Smells

Inappropriate Intimacy – one class depends too much on another class's internal details.

Message Chains – long chains of method calls across multiple objects.

Primitive Obsession – too many primitive types instead of domain objects.

Large Switch Statements – complex conditionals instead of polymorphism.

Speculative Generality – adding unnecessary abstractions for future (possible) needs.

Refused Bequest – inheriting methods that are not used or not applicable.

... and many other smells

Refactoring

Improve *internal* code structure while preserving *external* behavior.

Example: Splitting a large *CityManagement* class into *TrafficService*, *WasteService*, and *EmergencyService* classes.

Final Goal:
Create software that remains **reusable**, **flexible**, **understandable**, and **maintainable** as requirements change.

A Smarter Dhaka, Built on Good Design



- New services can be added easily —
- Modules work independently
- One part can change without breaking others
- Citizens get a unified smart-city experience

Concept	Short Explanation	Application in Dhaka Urban Management System (DUMS)	Example
MVC (Model–View–Controller)	Separates user interface, data, and request handling responsibilities.	Divides DUMS into **Model (urban data & logic), View (citizen/admin interfaces), Controller (request processing)**.	Citizen submits a waste complaint → **View:** Complaint Form → **Controller:** Processes request → **Model:** Stores complaint in database → View shows status update.
Information Hiding	Hides internal implementation details and exposes only necessary interfaces.	Protects internal operations of urban service modules.	Traffic module provides getTrafficStatus() API without exposing how traffic prediction is calculated.
Separation of Concerns (SoC)	Divides a system into independent parts, each handling a specific responsibility.	Separates traffic, waste, emergency, payment, and citizen service modules.	Waste collection scheduling is handled by **Waste Management Module**, not mixed with traffic operations.
Dependency Inversion Principle (DIP)	**High-level modules should depend on abstractions,** not specific implementations.	Urban services communicate through common interfaces.	EmergencyService depends on NotificationService interface rather than directly depending on SMS or Email systems.
Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	New urban services can be added without changing existing modules.	Adding a **Smart Parking Module** without modifying Traffic Management code.
Interface Segregation Principle (ISP)	Clients should not depend on unnecessary methods.	Provides specific interfaces for different users and services.	Traffic Officer uses TrafficControlInterface, while Waste Manager uses WasteCollectionInterface.
Coupling Management	Reduces dependency between modules to improve flexibility.	DUMS modules communicate through APIs/services instead of direct dependency.	Traffic Management receives road information from a shared City Data Service instead of directly depending on Waste Management.
DRY (Don't Repeat Yourself)	Avoids duplicate code and logic.	Common functions are reused across modules.	A single NotificationService is used for complaints, emergency alerts, and service updates.
Least Knowledge Principle	A module should know only necessary information about other modules.	Limits unnecessary communication between urban modules.	Complaint Module only requests service status; it does not access internal Waste Management operations.
Code Analysis	Evaluates code quality by identifying design problems.	Reviews DUMS modules for maintainability and complexity.	Detecting that Citizen Service class contains login, payment, complaint, and notification responsibilities.
Anti-patterns / Code Smells	Poor design practices that make software difficult to maintain.	Identifies problematic structures before they affect system quality.	A single **CityManagement class** controlling traffic, waste, emergency, and payment creates excessive responsibility.
Inappropriate Intimacy	One class depends too much on another class's internal details.	Prevents modules from accessing unnecessary internal data.	Traffic module directly modifying Waste Vehicle database records.
Message Chains	Long sequences of object calls indicate poor design.	Avoids complicated communication paths.	Citizen → Complaint → Department → Manager → Employee → Database (too many dependencies).
Primitive Obsession	Overuse of basic data types instead of meaningful objects.	Encourages proper domain modeling.	Instead of storing location as "23.8103,90.4125" string, create a **Location** class.
Large Switch Statements	Excessive conditional logic often indicates missing abstraction.	Replaces complex decision structures with polymorphism.	Instead of checking if(service=="Traffic"), if(service=="Waste"), create separate service classes.
Speculative Generality	Adding unnecessary features for possible future use.	Keeps DUMS simple and focused.	Avoid creating unused AI modules before actual requirements exist.
Refactoring	Improving internal code structure without changing external behavior.	Improves maintainability of DUMS while keeping services working.	Splitting a large CityManagement class into TrafficService, WasteService, and EmergencyService classes.
Final Goal	Build software that is reusable, flexible, understandable, and maintainable.	Ensures DUMS can evolve with future smart-city requirements.	Adding new IoT sensors or AI analytics without redesigning the entire system.

Sample Question

1. Explain the responsibilities of the Model, View, and Controller in the MVC Pattern.
2. What are the three components of the MVC Pattern?
3. Apply MVC to a university student registration system. Identify suitable responsibilities for the Model, View, and Controller
4. Design the architecture of an MVC-based Student Management System. Identify the major classes and explain how the Model, View, and Controller communicate.
5. Explain the DRY principle and the Principle of Least Knowledge.

Bloom's Level	Question
Remember	1. What are the three components of the MVC Pattern?
Understand	2. Explain the responsibilities of the Model, View, and Controller in the MVC Pattern.
Understand	3. Explain how the MVC Pattern supports separation of concerns.
Apply	4. Apply MVC to a university student registration system. Identify suitable responsibilities for the Model, View, and Controller.
Analyze	5. Analyze a program in which the Controller performs input validation, database operations, business logic, and HTML generation. Identify the design problems.
Evaluate	6. Evaluate the benefits of separating database operations, business logic, request handling, and presentation in an MVC-based application.
Create	7. Design the architecture of an MVC-based Student Management System. Identify the major classes and explain how the Model, View, and Controller communicate.

Bloom's Level	Question
Remember	1. Define an anti-pattern and a code smell. Give three examples of code smells.
Understand	2. Explain the meaning of the Large Class, Long Method, and Duplicated Code smells.
Understand	3. Explain the concepts of Primitive Obsession and Message Chains.
Apply	4. A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach.
Analyze	5. Analyze a system where the same switch statement for customer types appears in five different methods. Identify the code smell and explain why it creates maintenance problems.
Evaluate	6. Evaluate whether every occurrence of a code smell must always be refactored. Explain why the context of the software design should be considered.
Create	7. Create a refactored design for a system suffering from a Large Class and Duplicated Code. Identify how responsibilities should be redistributed among multiple classes.

Bloom's Level	Question
Remember	1. What are the DRY principle, Principle of Least Knowledge, Open/Closed Principle, and Information Hiding?
Understand	2. Explain how the DRY principle improves software maintainability.
Understand	3. Explain why excessive coupling between classes can make software difficult to maintain and modify.
Apply	4. Consider a class that directly accesses several internal objects of another class through a long chain of method calls. Identify the relevant design principle and suggest how it could be improved.
Analyze	5. Analyze the following situation: a superclass provides five methods, but a subclass only needs one of them and ignores the other four. Identify the possible code smell and explain the design problem.
Evaluate	6. Evaluate the following statement: **"Using more design patterns always results in better software design."** Support your answer with appropriate examples from Modules 1–3.
Create	7. Create a design-improvement plan for a poorly designed Java application containing a Large Class, Duplicated Code, Primitive Obsession, long methods, and tightly coupled classes. Explain which design principles, patterns, and refactoring techniques you would use.

Sample Question

1. A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach.
2. Analyze a system where the same switch statement for customer types appears in five different methods. Identify the code smell and explain why it creates maintenance problems.
3. Analyze a method containing 150 lines that performs validation, database operations, calculations, and report generation. Identify the possible smells and explain how they affect maintainability.
4. Evaluate whether comments are always beneficial in software development. Explain how excessive comments may sometimes indicate a design problem.
5. Create a refactoring plan for a Java application suffering from a Large Class, Long Methods, and Duplicated Code. Explain the sequence of improvements you would make.

Bloom's Level	Question
Remember	1. What are the three components of MVC? State the primary responsibility of each component.
Understand	2. Explain how separation of concerns is achieved through the MVC Pattern.
Understand	3. Explain the DRY principle and the Principle of Least Knowledge.
Apply	4. A Student Management System contains database queries directly inside its GUI classes. Apply the principles of MVC and separation of concerns to improve the design.
Analyze	5. Analyze a class that performs database access, user-interface formatting, validation, and business calculations. Identify the possible design problems and code smells.
Evaluate	6. Evaluate the statement: **"A large class is always an example of bad design."** Discuss the importance of context when identifying code smells.
Create	7. Design an MVC architecture for an online course-registration system. Identify the Model, View, Controller, and major responsibilities of each.

Bloom's Level	Question
Remember	1. Define Large Class, Long Method, Duplicated Code, and Primitive Obsession.
Understand	2. Explain why duplicated code can make software difficult to maintain.
Understand	3. Explain the difference between Divergent Change and Shotgun Surgery.
Apply	4. A program contains the same student-validation code in five different classes. Identify the code smell and propose an appropriate refactoring approach.
Analyze	5. Analyze a method containing 150 lines that performs validation, database operations, calculations, and report generation. Identify the possible smells and explain how they affect maintainability.
Evaluate	6. Evaluate whether comments are always beneficial in software development. Explain how excessive comments may sometimes indicate a design problem.
Create	7. Create a refactoring plan for a Java application suffering from a Large Class, Long Methods, and Duplicated Code. Explain the sequence of improvements you would make.

Bloom's Level	Question
Remember	1. Define Inappropriate Intimacy, Message Chains, Primitive Obsession, Speculative Generality, and Refused Request.
Understand	2. Explain why the Principle of Least Knowledge is useful for reducing unnecessary dependencies.
Understand	3. Explain why speculative generality can lead to unnecessary complexity in a software system.
Apply	4. A class accesses several objects through a long chain such as student.getDepartment().getTeacher().getOffice().getPhone(). Identify the possible design issue and suggest an improvement.
Analyze	5. Analyze a superclass that contains several methods that are not required by one of its subclasses. Identify the likely code smell and explain why it may indicate an inappropriate inheritance relationship.
Evaluate	6. Evaluate the statement: **"Design patterns should always be used to improve software design."** Discuss situations in which applying a design pattern could actually make a simple system unnecessarily complicated.
Create	7. Create a complete design-improvement strategy for a poorly designed Java application containing tight coupling, duplicated code, primitive obsession, long methods, and inappropriate intimacy. Identify the design principles, patterns, and refactoring techniques you would apply.